

Selection

Given a set S of n elements
 $\{x_1, x_2, \dots, x_n\}$ and an integer $1 \leq k \leq n$
 find the k -th element of S

rank $[k^{\text{th}}$ element is the k^{th} element of the
 k element sorted set S .

Obvious algorithm: Sort S and report the
 k^{th} element Time: $O(n \log n)$

Or build heap and
 use k extract min

Can we do it without explicitly sorting
 and preferably in linear time

Can be done using a fairly sophisticated
 algorithm based on the idea of Median-of-medians

$k = 1, n$ min / max finding external
 selection

?? { Can we do external selection faster
 than $n-1$ comparisons }

What about $k = 2$

We can find the k^{th} element using
 k passes through $S \Rightarrow kn$ comparisons

This worse than $O(n \log n)$ if $k = \omega(\log n)$

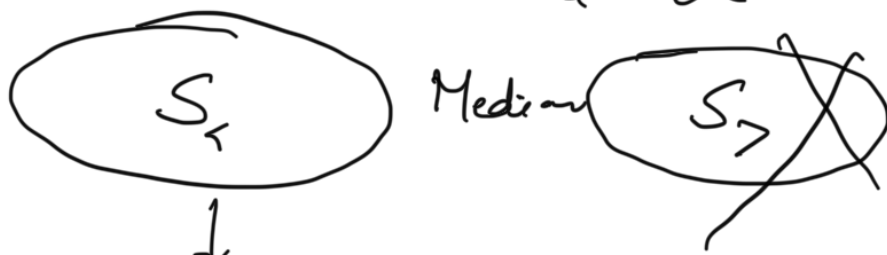
$k = \frac{n}{2}$ (median) $\Rightarrow O(n^2)$ alg.

[Suppose we know how to find the median]
 We have an oracle: that gives
 us median

$k = \frac{n}{4}$
 sorted median

$$\tilde{x}_1, \tilde{x}_2, \tilde{x}_3, \dots, \tilde{x}_{n/2}, \tilde{x}_{n/4}, \dots, \tilde{x}_n$$

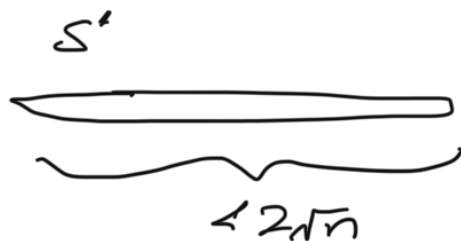
Use partitioning around the "median" and discard the second half



Find median : Done # comparisons n

$$k = \sqrt{n}$$

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots \rightarrow 2\sqrt{n}$$

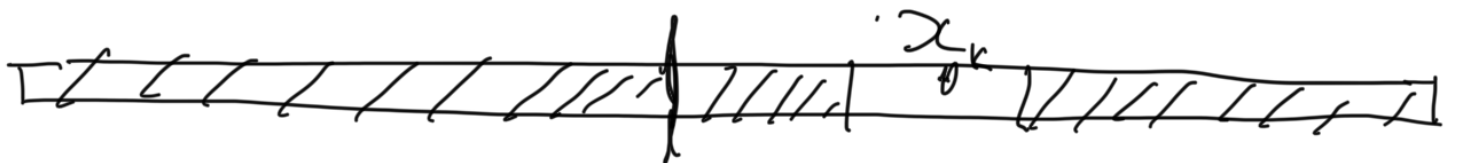


$$k = \sqrt{n}$$

Total Comparisons

$$n + \frac{n}{2} + \frac{n}{4} + \dots$$

We can repeat finding medians $< 2\sqrt{n}$ and discarding the one of the halves but we may have to adjust the value of k (depending on which side of the median)



What about the oracle cost?

$T(n)$: ^{worst case} cost of selecting k^{th} element among n elements

$$T(n) = n + T\left(\frac{n}{2}\right) + \text{hidden cost of oracle}$$

↑ cost of partition

Suppose, instead median, the oracle gave us an element whose rank is

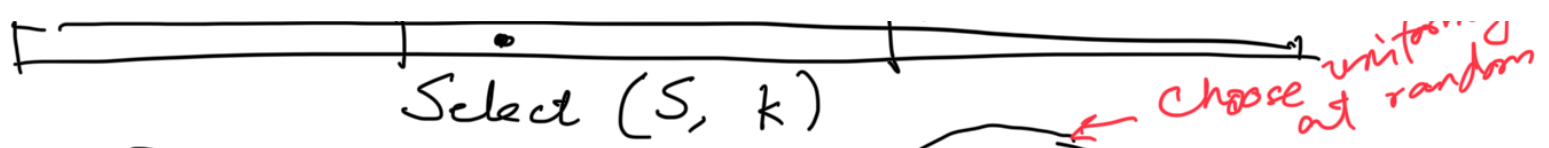
between $\frac{n}{4}$ and $\frac{3n}{4}$

$$\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_{n/4}$$

$$\tilde{x}_{3n/4}$$

$$\tilde{x}_n$$

...



1. Partition around the "pivot" element
 (S_L, S_R) Then we know the exact rank of the pivot, say r

2. If $k < r$ Select (S_L, k)
 $k > r$ Select ($S_R, k-r$)
 $k = r$ Return the pivot as the answer

Analysis

$$T(n) \leq \overset{\substack{\uparrow \\ \text{partitioning}}}{n} + (\text{recursive call}) T\left(\frac{3n}{4}\right)$$

Probability of picking a "good splitter" is $\frac{1}{2}$ (with prob $\frac{1}{2}$ we could fail)
 $\frac{n}{4} < r < \frac{3n}{4}$

If the splitter is "not good", then we repeat

This is similar to tossing a fair coin repeatedly until we get the first head

X : r.v. to represent the # of trials before we see the first "success"

$E[X]$ expectation For geometric distribut

$$E[X] = \frac{1}{p} \quad p: \text{prob of success}$$

$$= 2 \quad \text{if } p = \frac{1}{2}$$

If T is the running time of the "Randomized" Select algorithm, T is a random

$$E[T] = E[T_0 + T_1 + T_2 + \dots + T_m]$$

T_i : expected time for the i^{th} level of recursion

$$= \sum E[T_i] \quad \text{from linearity of expectation}$$

$$E[T_i] = 2 \cdot \left(\frac{3}{4}\right)^i n$$

$$E[T] = O(n) = cn$$

From Markov inequality

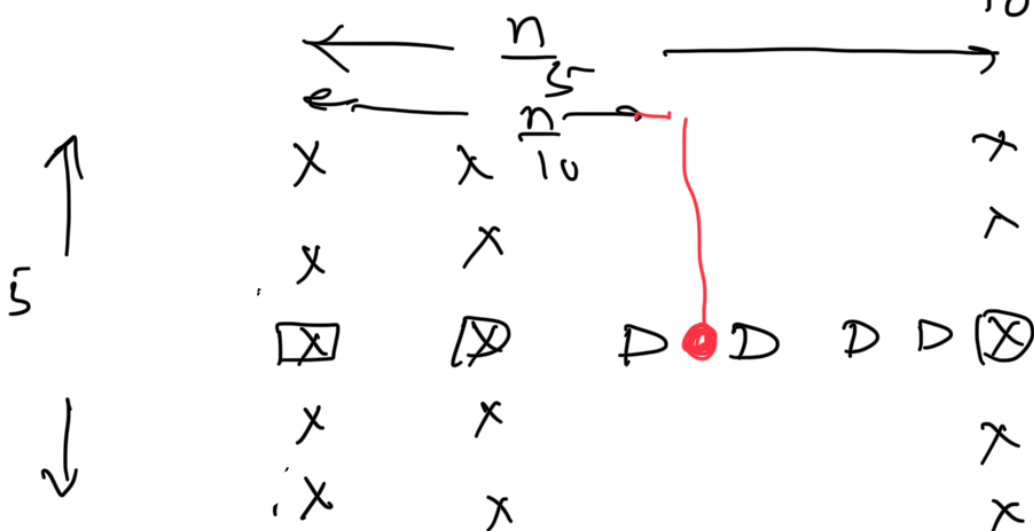
$$Pr[T > \alpha cn] < \frac{1}{\alpha}$$

$$\text{using } Pr[X > \alpha \cdot E[X]] < \frac{1}{\alpha}$$

Median of median algorithm

Is to choose a splitter

$$\text{having rank } \frac{3n}{10} \leq r \leq \frac{7n}{10}$$



MoM satisfies - this

finding MoM

$$T(n) = O(n) + T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right)$$

Cost of selection Sorting column recursive partitioning

The MoM algorithm
 $MoM(n, k)$

1. Make Groups of 5 elements and sort each group
 $\left(\frac{n}{5} \text{ groups}\right)$ Time: $\frac{n}{5} \times \text{time to sort 5 el.}$
 $\therefore O(n)$
2. Let the median of each group G_i be denoted by m_i .
 $\text{MOA} \left(\frac{n}{5}, \frac{n}{10}\right)$
 $\text{Time: } T\left(\frac{n}{5}\right) \leftarrow$ Find the median of m_i 's recursively, say M
3. Use M as the splitter for the original set of S elements - $S_{< M} \ S_{>}$ and partition S
4. Depending on the value of k, n recursively call $(S_{<}, k)$ or $(S_{>}, k')$
 $T(\quad, k)$ (Stop if $\text{rank}(M) = k$)

Claim $T(n) = O(cn)$

Where c is a constant

Verify the same

Approximate Median: An element in the given set of n elements that has rank between $[\alpha n, (1-\alpha)n]$ where α is some constant $0 < \alpha < 1$